TRESATA TAKE-HOME ASSIGNMENT

Auditable Covenant Compliance: compile the contract once, compute it deterministically every quarter

Domain: commercial-loan financial-covenant monitoring. **Jurisdiction:** US commercial credit agreements, using public SEC EDGAR filings. For another jurisdiction the definitions and amendment conventions would change; the architecture would not. **Evidence:** the agreement and amendment discussed below are real public documents; all borrower financial figures in the demonstration are synthetic. Cost figures are my estimates, not sourced industry benchmarks.

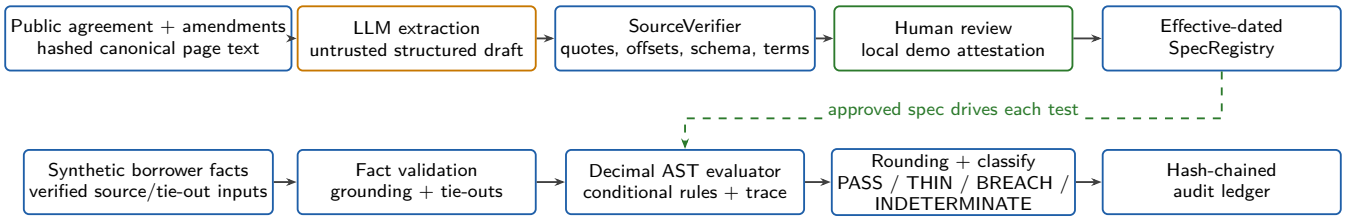
What surprised me. The ratio math is straightforward; the contractual definition is not. "Consolidated EBITDA" can contain bespoke add-backs and sub-caps; cash netting can be conditional; thresholds can step down; and an amendment can change which definition applies to a test date. A generic template misses these details, while an LLM-generated ratio is difficult to reproduce and audit.

Problem statement

Who	Covenant-monitoring analysts in credit administration at a mid-market lender, with a credit officer accountable for findings.
Task	For each borrower and reporting period, identify financial covenants in a long credit agreement, apply amendments, extract financial inputs from statements and compliance certificates, recompute ratios under the contract's definitions, and flag breaches or thin headroom.
Frequency	Quarterly (sometimes monthly). Illustrative scale: 400 borrowers, or about 1,600 tests per year. Certificates may cluster after quarter-end, creating a short review peak.
Cost today	Working estimate: 4.5 hours per test, or 7,200 hours annually. At an assumed loaded rate of $90/hour, this is about $648,000 (approximately $650K). These assumptions require validation with practitioners. The larger risk is a late-detected breach or a false pass.
Out of scope	Waivers and reservation-of-rights decisions, legal notices, pricing and rating, non-financial covenants, equity-cure calculations, pro-forma acquisition adjustments, and real client data. The system provides decision support, not legal advice.

Solution design

Core idea. Compile each agreement version into a typed, source-grounded `CovenantSpec`. A human reviews and approves that specification. A deterministic engine then evaluates verified borrower facts against the approved specification each quarter. The LLM interprets unstructured contract language; it does not calculate the financial ratio or decide covenant status.



Why AI is needed. Contract definitions are bespoke natural language with nested caps, conditions and cross-references. Retrieval and an LLM can help interpret that language and map it into a typed representation. Exact arithmetic, validation, rounding and classification remain ordinary code. This division keeps probabilistic interpretation separate from auditable calculations.

Knowledge and lifecycle. Contract meaning lives in a versioned, approved `CovenantSpec`; source evidence lives in hash-pinned documents; bank-specific thin-headroom and sampling rules live in `BankPolicy`. An amendment creates a new effective-dated specification; a restatement invalidates affected facts and dependent results; a revoked specification blocks the periods it governed until a replacement is approved. Historical records are not silently overwritten. A known but unread amendment blocks approval.

Audit lineage. Each result links to its ratio, AST trace, fact IDs, exact source quotations and offsets, document hash, specification hash, policy hash and engine version. The ledger records approvals, results and lifecycle events so a result can be reconstructed from its sealed inputs.

Workflow and implementation

The implementation has two distinct paths. Contract extraction creates an untrusted draft that must pass provenance and structural checks before a human can approve it. Quarterly computation accepts only verified facts and an approved, effective specification.

Step	State	Actor	Operation and exit condition
A1	INGESTED	Code	Hash document and preserve canonical page text and offsets. The public source corpus is reproducible from SEC EDGAR.
A2–3	EXTRACTED	LLM or recorded fixture	Identify covenant language and produce structured JSON. Model output is untrusted; software derives document hashes and offsets from the cited page and quotation.
A4	VERIFIED	Code	Independently verify quotations, document hashes, defined terms, AST structure, constants, threshold schedules and known amendment coverage. Reject unresolved or unsupported claims.
A5	REVIEWED	Human	Display AST-generated formula, source quotations, threshold schedule, scope notes and unread terms. Reviewer explicitly approves or rejects the spec.
A6	REGISTERED	Code	Verify approval attestation and register the approved specification with its effective date.
B1–2	FACTS	Code and demo data	Load synthetic financial facts, verify source spans and calculate tie-outs. Missing or unverified mandatory facts cannot produce PASS.
B3–5	COMPUTED	Code	Evaluate the closed AST with Decimal arithmetic, apply contractual rounding, calculate headroom and classify the result.
B6–7	SEALED, REVIEW	Code and human	Store result and trace in the hash-chained ledger; route breach, thin-headroom and indeterminate cases for human review.

Implemented components

Component	Role and enforced invariants
`SourceSpan` / `SourceVerifier`	Stores document hash, page, offsets and verbatim quote. Spans carry no self-asserted verified flag; the trusted verifier checks document identity and exact text before sealing.
`Expr` (AST)	Closed expression set: `Lit, Input, TermRef, Sum, Diff, Mul, Div, MinOf, MaxOf, IfGt`. The LLM supplies data, never executable code.
`CovenantSpec`	Numerator, denominator, defined terms, inputs, threshold steps, rounding and scope notes. Rejects undeclared inputs, unresolved or cyclic terms and invalid schedules.
`BankPolicy`	Holds operational policy such as thin-headroom threshold, sampling percentage and OCR confidence requirements; policy is separately versioned and hashed.
`FinancialFact`	Uses Decimal values, currency, period, scale, source evidence and grounded tie-out components. Tie-out success is derived, not supplied by the extractor.
Seal + approval	Bind verified content and approval metadata. The development signer uses local HMAC; it demonstrates tamper detection, not real-world identity authentication.
Registry + ledger	Preserve effective-dated versions, approval events, results and hash-chain history. Restatements mark dependent results stale; amendments create a new specification version.
Test result	Revalidates rounding, arithmetic, trace, headroom and classification. Staleness and previous-hash linkage are ledger concerns, not mutable result fields.

What the prototype demonstrates, and what it does not

Deterministic core: implemented. The AST evaluator performs Decimal arithmetic under an explicit 50-digit context, supports conditional rules and creates a node-level trace. The pipeline validates facts, rounds and classifies results. The registry and ledger preserve version and audit history.

Extraction path: implemented, but not validated with live inference. The OpenRouter provider handles timeouts, rate limits, truncation checks and requests without tools; its HTTP behaviour was tested against a fake transport. No live OpenRouter API call was made because no API key was available. The recorded provider replays `data/extraction/viad_reference_response.json`, a reference specification manually written from the public agreement. The CLI labels this *NOT A LIVE MODEL CALL*; it is not evidence of LLM extraction accuracy.

Human approval: simulated. The demonstration uses reviewer identifier `demo.reviewer@example.com` and a local development HMAC signer. This proves that the attestation record is not altered, but does not authenticate a real credit officer. Production requires an identity-backed approval service and KMS/HSM-managed keys. No real credit officer signed the demonstration. Signed rejection records are saved locally; the CLI does not yet append them to the audit ledger.

Reproducibility. The public agreement pages are committed as hash-pinned canonical text; `tools/build_sources.py --check` re-derives them from EDGAR. The reference specification contains 7 defined terms, 20 inputs and 4 threshold steps.

Public agreement and worked example

Source agreement. Viad Corp (now Pursuit Attractions and Hospitality), CIK 0000884219. The original credit agreement is dated 30 July 2021 (SEC accession 0001564590-21-039835); its first amendment is dated 23 March 2022 (accession 0000950170-22-004478). The prototype uses the agreement's EBITDA and net funded indebtedness definitions (§1.01), cash-netting cap (§1.01), rounding rule (§1.04), and amended leverage schedule (First Amendment §1, restated §7.10(c)).

Public filings: Original Credit Agreement, SEC EDGAR | First Amendment, SEC EDGAR. Filing-rendered page references used in the reference extraction: EBITDA and net funded indebtedness, pp. 17–19; unrestricted cash, p. 62; rounding, p. 64; amended covenant levels, pp. 2–3.

Synthetic borrower calculation

All borrower figures below are synthetic and expressed in USD millions. They demonstrate the contract mechanics, not an actual Viad borrower result. The example assumes no Acquisition Period is continuing.

Calculation	Contractual operation	USD m
EBITDA before clause (d)	$70 + 25 + 20 + 40 + 18 + \min(3, 5) + 0 + 6 + 4 + 0 - 2$	184.0
Clause (d) add-back	$\min(40,\ 20.0\% \times 184.0) = \min(40,\ 36.8)$	36.8
Consolidated EBITDA	$184.0 + 36.8$	220.8
Netted unrestricted cash	Revolver borrowings $25 > 10$; cash allowed is $\min(120,\ 50)$	50.0
Net funded debt	$(800 + 80 + 5) - \min(30,\ 50) - 0 - 50$	805.0

$$\text{Leverage ratio} = 805/220.8 = 3.645833\ldots \longrightarrow 3.646\times$$

Test date	Limit	Ratio	Headroom	Result
31 Dec 2022	4.75x	3.646x	23.24%	PASS
30 Jun 2023	4.00x	3.646x	8.85%	PASS_THIN_HEADROOM (illustrative 10% BankPolicy threshold)

Scope assumption. The amendment also provides alternative leverage levels during an Acquisition Period; those alternatives are not modelled. The examples assume no Acquisition Period is continuing. A production workflow must resolve this condition from reliable borrower/event data before selecting the threshold; it must not silently assume the standard schedule when the condition is unknown.

Validation and risk analysis

Test evidence reported by the project. 246 tests pass offline and a fresh clone reproduces them; mypy is clean across 25 source files. All 28 deliberate mutation checks are caught. No live OpenRouter call has been made.

Risk	Failure mode	Control / limitation
Semantic error	A real quotation is interpreted incorrectly; a consistent AST can still encode the wrong meaning.	Source verification proves provenance, not interpretation. Human review is essential; independent historical back-testing is future work.
Outdated or incomplete terms	An amendment or side letter is missing, or a known amendment is unread.	Known unread amendments block approval. A complete, externally confirmed amendment register remains a production requirement.
Adversarial input	Prompt injection, hidden text or fabricated figures.	Model output is untrusted; no tools or executable code are accepted. Quotes and tie-outs are checked. OCR and hidden-text comparison are not implemented.
Data and liability	Borrower statements contain material non-public information; a demo identity is mistaken for an accountable officer.	Prototype uses public contracts and synthetic data. Local HMAC demonstrates integrity, not identity. Production needs approved data residency, authenticated roles and KMS/HSM signing.

Alternatives considered and next steps

I rejected (1) a single long-context LLM call returning the ratio, because arithmetic is not reliably reproducible and source lineage is difficult to enforce; (2) per-quarter RAG chat, because it can reinterpret definitions across periods; and (3) rules/templates alone, because bespoke definitions, conditions and amendments are too varied. Deterministic rules remain in the AST execution engine.

Before investing further. Validate the 4.5-hour estimate and amendment-completeness process with lenders; measure live extraction on 10+ public agreements against independently reviewed specifications; and confirm tooling gaps, identity, data-residency and integration requirements. The prototype code and tests are in the public repository github.com/Siddharth-6/covenant-audit.

Appendix: What I added beyond the AI

Where AI helped. I used Claude in Cursor to research the domain, challenge the architecture, draft typed models, implement the deterministic engine and extraction workflow, generate tests, and improve this brief. Claude wrote nearly all of the code. Excerpts from the prompts I gave it, verbatim from my chat logs:

1. *"Maintain strict separation between probabilistic nodes (LLM document ingestion, unstructured text extraction, semantic RAG) and deterministic execution engines (Pydantic validation, financial ratio math, policy rules engines)."*
2. *"Do not assume that the existing implementation is correct merely because tests pass."*
3. *"The LLM should interpret the contract, but the deterministic engine must remain the sole authority for financial calculations and covenant classification."* and *"Do not simulate an LLM call while claiming that a real model was used."*

What the AI got wrong, and what I required it to fix.

- *Unsourced precision.* Claude proposed precise-looking cost figures (4.5 hours per test, $650K a year). They are assumptions; I kept them labelled as my estimates and made practitioner validation a next step.
- *Defects the AI disclosed.* Claude's first implementation hardcoded the 10% thin-headroom threshold, trusted a caller-supplied tie-out `passed` flag and a self-declared `verified` flag, and validated results in one direction only. After reading its own limitations list, I made fixing each a requirement and told it not to weaken existing tests; they became a versioned `BankPolicy`, derived tie-outs, a trusted verifier with sealed content, and a result validator that re-derives classification.
- *A gap the AI found in its own fix.* While writing a prompt-injection test, Claude noticed tie-out components were still the extractor's word; each must now be a quoted, summed source line.
- *Designs ruled out.* LLM-generated Python for calculations (I instructed that the LLM must not generate executable code) and LLM self-reported confidence as a routing signal, because uncalibrated confidence correlates poorly with mathematical precision.
- *Bugs the tests exposed, fixed by the AI.* A canonicaliser that mis-paired nested quotes, a clause search that ran out of budget before the cash definition, and a false-positive "constant denominator" check. An old import-ban test that the new download code correctly broke was narrowed with a documented reason, not deleted.

What came from me. I set the problem (commercial-loan covenant monitoring) and the constraints: a strict boundary between probabilistic extraction and deterministic computation; the four audit questions (where knowledge lives, lineage, weakest link, three rejected alternatives); and a four-page limit with this appendix. I required a real public SEC agreement rather than invented clauses, and that a missing or unread amendment never be treated as absent. I directed the hardening phase (listing the defects to fix) and required that no replayed or simulated model call be presented as live, that the development signer be kept apart from production KMS signing and not described as identity authentication, and that implemented, simulated, production and unimplemented items be kept separate. I then reviewed this brief against the repository and had claims I could not stand behind corrected. The compile-once architecture, the choice of the Viad agreement and the conditional AST operation were proposed by Claude; I reviewed and accepted them.

Important disclosure about the demonstration. The repository's `viad_reference_response.json` was manually written from the public agreement text. The recorded provider replays that reference and is explicitly labelled *NOT A LIVE MODEL CALL*. The OpenRouter adapter is implemented and tested against a fake transport, but no live API call was made because no API key was available. The demo approval by `demo.reviewer@example.com` is simulated and is not a real credit officer's sign-off. The prototype therefore demonstrates the extraction, verification, review and calculation interfaces, but does not establish live-model extraction accuracy or production identity authentication.

One concept I can defend without AI: determinism is not semantic correctness. For the synthetic example, the contract-capped EBITDA is $184.0 + \min(40.0,\ 0.20 \times 184.0) = 220.8$ million. Net funded indebtedness is 805 million, giving a ratio of $805/220.8 = 3.645833\ldots$, rounded to $3.646\times$ under the agreement's three-decimal, ties-up rule. The engine can reproduce this result exactly from the approved AST and verified inputs. But it cannot prove that the AST correctly represents the legal meaning of the agreement. A real quotation can be grounded and still be misunderstood. That is why semantic review, amendment completeness and independent validation remain critical.

All borrower amounts and borrower scenarios in this brief are synthetic. Public contract sources are cited above. Prototype limitations are stated explicitly; no live LLM inference or real credit-officer approval is claimed.